Fault Tolerant Topic-based File Distribution Service

Design Document

IT305 Computer Networks Lab – Socket Programming Project

Lab Group 43 + Lab Group 6

Implementation platform: C (gnu11), POSIX threads, TCP/IPv4 sockets, Linux

#	**Full Name**	**Roll Number**	**Contribution**
1	Madhav Chauhan	202401106	Implemented and documented Part I, including single-threaded and multi-threaded server modes and basic client-server communication.
2	Dhruva Bhatiya	202401029	Worked on Part I message handling, topic/file management, client-side file reception, and testing of single- and multi-threaded operation.
3	Tasvi Bhalani	202401027	Implemented and tested Part II Case 1, including probabilistic fault injection, connection failure handling, retry logic, and complete topic retransmission.
4	Tanvi Kakadiya	202401078	Implemented and documented Part II Case 2 session management, including checkpoints, RESUME/RESUME_OK handling, recovery, and partial file transfer.
5	Bhagy Parmar	202401137	Worked on Part II Case 2 Enhanced, including parallel TCP connections, dynamic work queue, concurrent file transfers, and enhanced recovery mechanisms.
6	Kanika Pal	202401082	Worked on Part II Case 2 Enhanced performance improvements, fault recovery, zero-copy file transfer, testing, and performance evaluation.

Contents

1 Overview, assumptions and command-line interface **4**

1.1 The four versions 4

1.2 Assumptions 4

1.3 Command-line interface (as required for the auto-grader) 4

1.4 Summary of key design decisions 5

2 Server data structures: mapping topics to directories **5**

2.1 Construction, lookup and concurrency 6

3 Client–server message formats **6**

3.1 Common framing header (16 bytes) 7

3.2 Message types 7

3.3 Payload layouts 7

3.4 Error handling common to all cases 8

3.5 Part I – message exchange 9

3.6 Part II Case 1 – message exchange (no session management) 9

3.7 Part II Case 2 – message exchange (with session management) 10

3.8 Part II Case 2 Enhanced – message exchange (parallel connections) 10

4 Fault injection mechanism **11**

4.1 When the failure decision is made 11

4.2 How the failure decision is made 11

4.3 How the connection is interrupted 12

4.4 Expected effect of p 12

5 Session management protocol (Case 2) **13**

5.1 Sequence and types of messages 13

5.2 Message formats used by the session protocol 13

5.3 Information stored by the server 14

5.4 Information stored by the client 14

5.5 Validation of a checkpoint and resumption 15

5.6 Checkpoint update frequency 15

6 Planned performance enhancements **15**

6.1 Socket mode 16

6.2 Parallel TCP connections 16

6.3 Other enhancements (all parts where applicable) 17
6.4 Experimental setup 17
6.5 Part I experiments 18
6.6 Part II experiments 18
6.7 Metrics 18

7 Additional features beyond the minimum requirements **19**

8 Code structure **20**
8.1 Module and file structure 20
8.2 Key functions 20
8.3 How the single-threaded, multi-threaded and fault-tolerant logic fit together . . . 21
8.4 Build 22

9 Design thought process: alternatives considered **22**

1 Overview, assumptions and command-line interface

The system is a TCP file server. It keeps a set of *topics*, one per sub-folder of the Animals-10 dataset (cane, cavallo, elefante, farfalla, gallina, gatto, mucca, pecora, ragno, scoiattolo). A client sends GET `<topic>`. The server sends every file in that topic's directory, and the client writes them into `output_dir/<topic>/`. We implement four versions. Each one builds on the previous one and uses the same message format.

1.1 The four versions

Version	Folder	Behaviour
Part I	`Part1/`	Topic server in two modes. Single-threaded mode serves clients one after another. Multi-threaded mode serves them concurrently, one thread per client.
Part II – Case 1	`Part2_Case1/`	Multi-threaded server with probabilistic fault injection. No transfer state is kept, so after a failure the client restarts the topic from the beginning.
Part II – Case 2	`Part2_Case2/`	Case 1 plus session management. Client and server keep checkpoint state, and after a failure the transfer resumes from the last checkpoint (file + byte offset).
Part II – Case 2 Enhanced	`Part2_Case2_Enhanced/`	Case 2 plus parallel TCP connections per topic, a shared work queue and zero-copy `sendfile()`.

1.2 Assumptions

- Linux lab machines with gcc and glibc, TCP over IPv4, and a switched LAN. One machine runs the server and the other two run multiple clients each.
- `topics_root_dir` contains one sub-folder per topic, and the folder name is the topic key. Regular files directly inside a topic folder are served. Hidden files and nested sub-folders are ignored.
- No path, port or IP address is hard-coded. All of them come from the command line.

1.3 Command-line interface (as required for the auto-grader)

Program	Required invocation	Optional trailing arguments (default)
Server, Part 1	`./server <port> <topics_root_dir>`	`[single|multi]` (multi)
Server, Part 2 (all cases)	`./server <port> <topics_root_dir> <failure probability>`	`[seed]` (random). A fixed seed makes fault experiments repeatable.
Client (all parts)	`./client <server_ip> <server_port> <topic> <output_dir>`	Case 1: `[max_retries]` (100). Case 2: `[max_retries]` (20). Enhanced: `[connections] [max_retries]` (4, 20).

`make` in each of the four folders produces exactly two executables, `server` and `client`. The failure probability p must be a real number in $[0, 1]$. Invalid arguments print a usage line and exit with status 1. The client exits with 0 on success, 1 on a fatal error (e.g. unknown topic) and 2 when it gives up after `max_retries`.

1.4 Summary of key design decisions

Design question	**Decision**	**§**
Topic → directory map	Built once at startup. A sorted array of topics, each holding its absolute path and a sorted, cached list of files, so file indices are stable.	2
Message framing	Every message starts with a 16-byte header: magic, version, type, and a 64-bit payload length.	3
Single vs multi-threaded	One program. Multi mode starts a detached pthread per accepted client, plus a signal-handling thread.	1.3, 8
Fault decision	Made in the worker's file-send loop **before every 64 KB chunk**: one independent `drand48_r()` draw. If it is below p (the per-chunk fault probability), the server calls `shutdown()` + `close()`.	4
Case 1 recovery	The client reconnects and sends GET again, and the whole topic is re-sent.	3.6
Case 2 checkpoint	The client is authoritative: `.part` file size = byte offset, plus a `.session` record. The server keeps a session table and validates every RESUME.	5
Case 2 Enhanced	K = 4 parallel connections by default. A dynamic file queue, one file per FETCH. A failed file is re-queued with its offset.	6.2
Socket mode	Blocking sockets with threads and time-outs. A non-blocking `poll()` client is a measured variant.	6.1

2 Server data structures: mapping topics to directories

The server keeps a map from each topic key to the absolute path of its directory. It also caches that directory's file list, sorted by name. The map is built once at startup from `topics_root_dir`. A request is then answered without scanning the disk, and a file index always identifies the same file, which the Case 2 checkpoints rely on.

```
typedef struct {
    char     name[NAME_MAX + 1];   /* e.g. "OIP-a1b2.jpeg"                    */
    uint64_t size;                 /* bytes, from stat() at load time         */
} file_entry_t;

typedef struct {
    char          key[64];         /* topic key = folder name, e.g. "gatto"   */
    char          abs_path[PATH_MAX];  /* realpath() of the folder (map value) */
    file_entry_t *files;           /* sorted by name (qsort + strcmp)         */
    uint32_t      n_files;
    uint64_t      total_bytes;     /* sum of all file sizes                   */
} topic_t;

typedef struct {
    topic_t          *topics;      /* sorted by key -> bsearch() lookup       */
    size_t            n_topics;
```

```
    char                root[PATH_MAX];
    pthread_rwlock_t    lock;        /* protects re-scan for newly added topics  */
} topic_map_t;
```

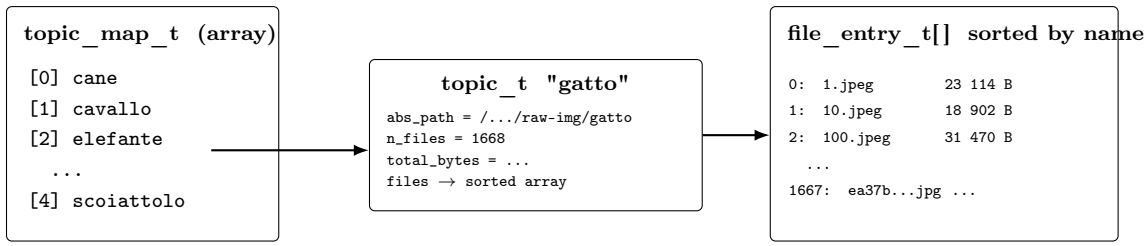


Figure 1: Topic map: topic key → absolute directory path → sorted file list.

Key (topic)	**Value (absolute directory path)**
cane	`<topics_root_dir, resolved by realpath()>/cane`
gatto	`<topics_root_dir, resolved by realpath()>/gatto`
`...` one entry per sub-folder `...`	`...`

2.1 Construction, lookup and concurrency

- **Construction:** the server reads `topics_root_dir` with `opendir()`/`readdir()`. Every sub-folder (checked with `stat()` and `S_ISDIR`) becomes a `topic_t` whose value is its `realpath()`. Each topic folder is scanned once. Regular files are stored with their sizes and sorted by name, and the topic array is sorted by key.
- **Lookup:** `topic_find(map, key)` first tries an exact, case-sensitive `bsearch()`. If that fails it tries a case-insensitive match, because the assignment's table writes "Cane" while the folder is "cane". An unknown topic gets `ERROR 404`.
- **Concurrency:** after startup the map is only read, so worker threads use it without contention. They take a read lock on the `pthread_rwlock_t`. The write lock is taken only when the map is re-scanned to add a newly created topic folder (§7).

3 Client–server message formats

TCP delivers a byte stream without message boundaries, so every message is framed with a fixed-size header that carries its type and its payload length. The receiver reads exactly 16 header bytes, then exactly `payload_len` bytes. All four versions share this header. Each later version only adds message types. §3.1–3.3 define the formats and §3.5–3.8 give the exchange for each case.

3.1 Common framing header (16 bytes)

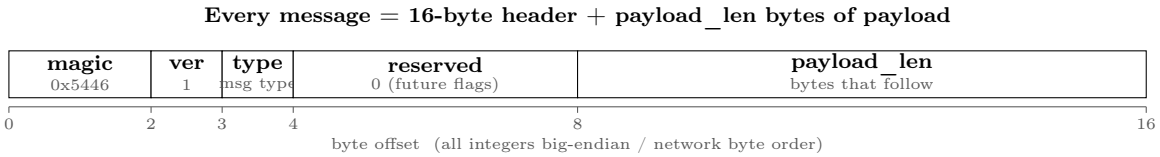


Figure 2: Header used by every message in every part.

- All integers are sent in network byte order (`htons`, `htonl`, `htobe64`) and converted back on receipt. Structures are never sent raw. Each field is packed into a byte buffer individually, which avoids problems with compiler padding and byte order.
- The receiver rejects a header whose `magic` or `version` is wrong, or whose `payload_len` exceeds 64 KB for a control message. It sends `ERROR 400` where possible and closes the connection.
- Every read uses `recvall()` and every write uses `sendall()`. These loops continue until the full length is transferred and retry on `EINTR`. `recv() == 0` before a message is complete means the peer closed the connection, which is treated as a connection failure.

3.2 Message types

Code	Name	Direction	Used in
0x01	GET	client → server	Part I, Case 1, Case 2 (first connection)
0x02	RESUME	client → server	Case 2 (reconnection)
0x03	MANIFEST_REQ	client → server	Case 2 Enhanced
0x04	FETCH	client → server	Case 2 Enhanced
0x10	TOPIC_BEGIN	server → client	Part I, Case 1, Case 2
0x11	FILE_HDR (+ raw file bytes)	server → client	all
0x12	FILE_END	server → client	all
0x13	TOPIC_END	server → client	Part I, Case 1, Case 2
0x14	RESUME_OK	server → client	Case 2
0x15	RESUME_REJECT	server → client	Case 2, Case 2 Enhanced
0x16	MANIFEST	server → client	Case 2 Enhanced
0x7F	ERROR	server → client	all

3.3 Payload layouts

Sizes are in bytes. *str* = raw bytes without a terminating NUL, always preceded by a length field.

Message	Payload fields (in order)	Notes
`GET`	ASCII text `"GET <topic>"`	The request has exactly the form the assignment specifies, carried inside the standard header. Topic length is 1–63 bytes.
`TOPIC_BEGIN`	session_id[16] · file_count u32 · total_bytes u64	session_id is all zeros in Part I and Case 1. In Case 2 it is 16 random bytes (`getrandom()`).
`FILE_HDR`	file_index u32 · file_size u64 · start_offset u64 · name_len u16 · name str	Immediately followed by exactly **file_size − start_offset** raw file bytes, which are not counted in payload_len. start_offset is 0 except when resuming.
`FILE_END`	file_index u32	Marks the file complete.
`TOPIC_END`	file_count u32 · total_bytes u64	The client checks both totals against what it received.
`RESUME`	session_id[16] · file_index u32 · offset u64 · topic_len u8 · topic str · name_len u16 · name str	file_index = first incomplete file. offset = bytes of that file already stored by the client.
`RESUME_OK`	file_index u32 · offset u64	The server confirms the resume point.
`RESUME_REJECT`	reason u16	1 unknown topic · 2 index out of range · 3 name mismatch · 4 offset > file size · 5 checkpoint ahead of the data sent.
`MANIFEST_REQ`	topic_len u8 · topic str · connections u8	connections = K, the number of parallel connections the client will open.
`MANIFEST`	session_id[16] · file_count u32 · total_bytes u64 · then per file: file_size u64 · name_len u16 · name str	A file's index is its position in the list.
`FETCH`	session_id[16] · file_index u32 · offset u64 · topic_len u8 · topic str · name_len u16 · name str	Same fields as RESUME, so each request is self-contained. Reply: FILE_HDR + data + FILE_END.
`ERROR`	code u16 · msg_len u16 · msg str	400 malformed request · 404 unknown topic · 500 file unreadable · 503 server busy (thread limit reached).

For `FILE_HDR`, `payload_len` covers only the file metadata; the subsequent file data is a separate length-delimited byte stream containing exactly `file_size − start_offset` bytes.

3.4 Error handling common to all cases

- Unknown topic: the server replies `ERROR 404` and closes. The client prints the message and exits with status 1.
- Malformed header or payload: `ERROR 400` and close. An unreadable file gives `ERROR 500`, and the thread limit gives `ERROR 503`.
- The client rejects any server-supplied file name containing '/', '..', a leading '.' or control characters (§7).

3.5 Part I – message exchange

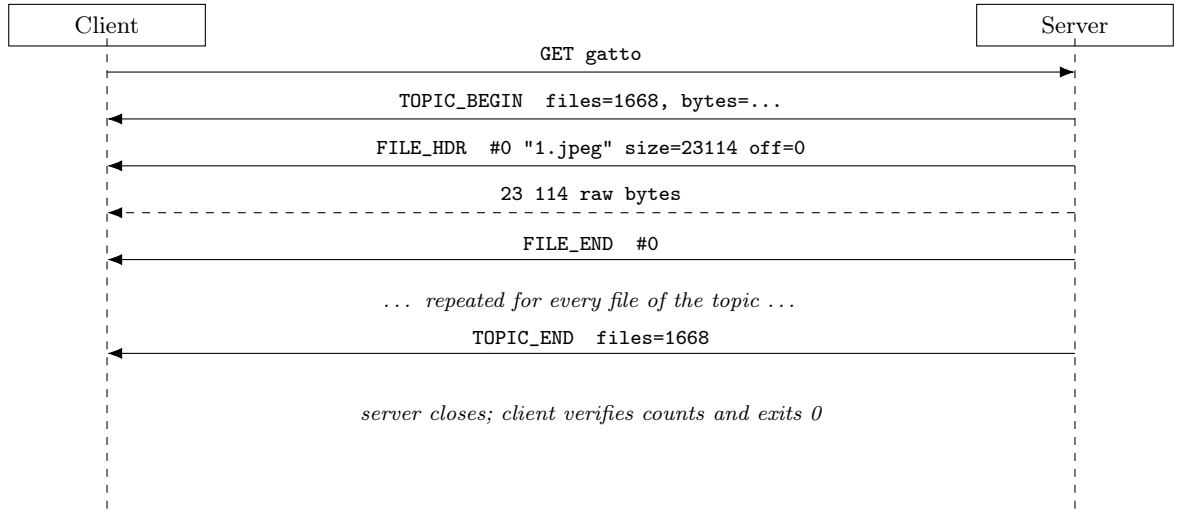


Figure 3: Part I. The single- and multi-threaded servers use identical messages; they differ only in how connections are scheduled (§8.3).

3.6 Part II Case 1 – message exchange (no session management)

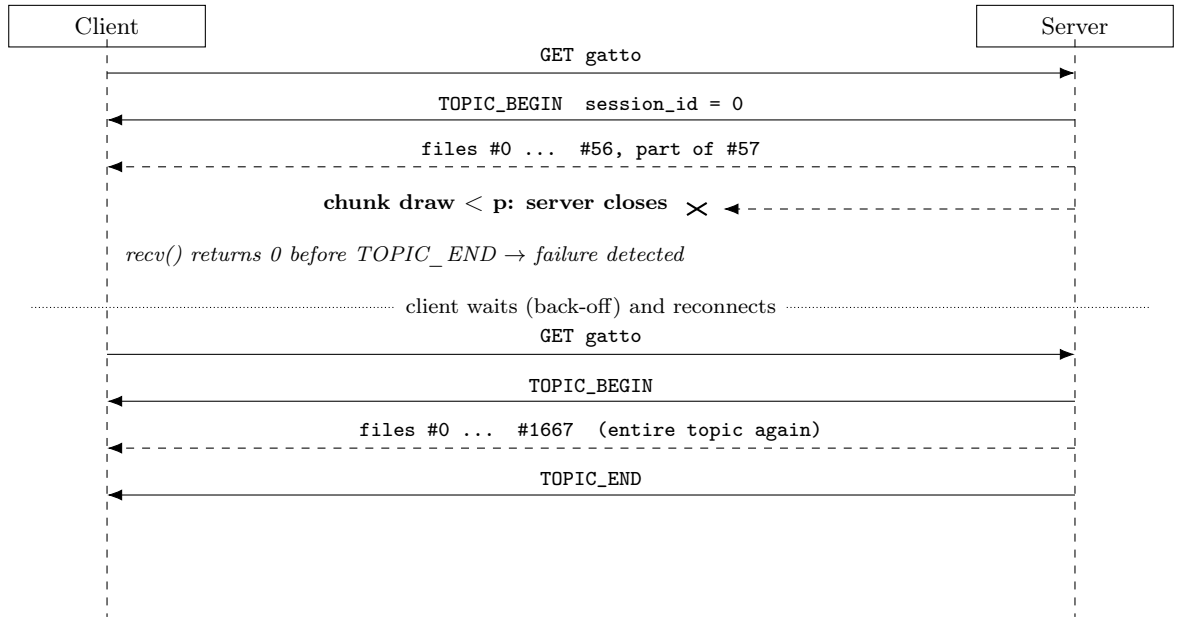


Figure 4: Case 1. After a failure the whole topic is transmitted again.

Case 1 uses the same messages as Part I. Neither side keeps transfer state. The client opens every output file with `O_TRUNC`, so a restart overwrites any partial data. Back-off between attempts is 50 ms, doubling up to 1 s. After `max_retries` attempts the client exits with status 2.

3.7 Part II Case 2 – message exchange (with session management)



Figure 5: Case 2. After a failure only the bytes not yet stored are transmitted.

3.8 Part II Case 2 Enhanced – message exchange (parallel connections)

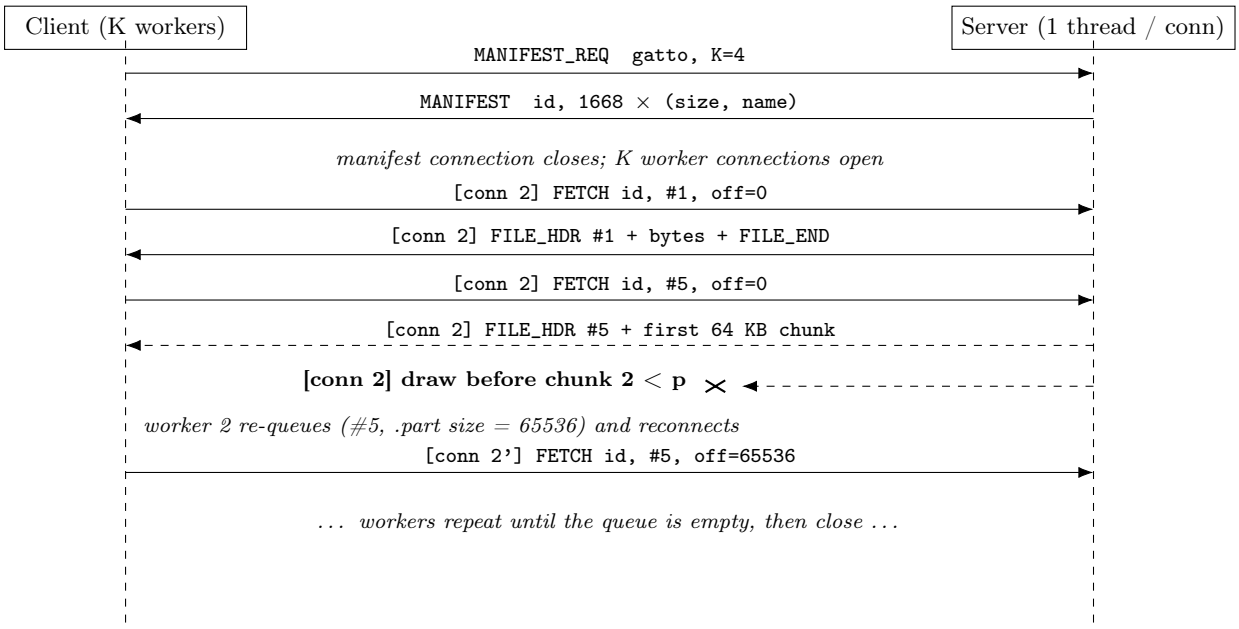


Figure 6: Case 2 Enhanced. Each of the K connections fetches one file at a time from a shared queue.

4 Fault injection mechanism

The server interrupts active connections at random, and the `<failure probability>` argument p controls how often. **p is the probability that the connection fails before any given 64 KB chunk of file data.** Every chunk gets its own independent random decision, so p is a **per-64-KB-chunk** fault probability. It is not a per-connection or per-session probability.

4.1 When the failure decision is made

The decision is made inside the server worker's **file-send loop**, immediately **before every 64 KB chunk** of file data is sent. The last chunk of a file may be smaller than 64 KB, and a 0-byte file has no chunk and therefore no draw. Headers and control messages (TOPIC_BEGIN, FILE_HDR, FILE_END, TOPIC_END, RESUME_OK, MANIFEST) are never preceded by a draw, so every injected failure happens during a file transfer.

The same rule applies in every Part II case. In Case 1 and Case 2 the connection's worker thread draws before each chunk of the topic. In Case 2 Enhanced each of the K connections is served by its own thread, which draws before each chunk it sends. A reconnection is a new connection, and its chunks are drawn for in the same way.

4.2 How the failure decision is made

```
/* per-connection state, owned by exactly one worker thread */
struct drand48_data rng;      /* srand48_r(seed ^ conn_id, &rng) when the connection starts;
                                 without a [seed] argument: time ^ thread id ^ conn_id     */

/* file-send loop inside send_file_range() */
while (off < file_size) {
    size_t n = MIN(CHUNK_SIZE /* 65536 */, file_size - off);
    double r;
    drand48_r(&c->rng, &r);               /* one independent draw per 64 KB chunk      */
    if (r < c->p) {                       /* fault triggered                           */
        log_fault(c, file_idx, off, chunk_idx);
        shutdown(c->fd, SHUT_RDWR);
        close(c->fd);
        return FAULT_INJECTED;            /* worker thread ends; server keeps running */
    }
    send_chunk(c, in_fd, &off, n);        /* sendfile() loop, advances off by n        */
    c->bytes_sent += n;  chunk_idx++;
    session_update(c, file_idx, off);     /* Case 2 and Enhanced only                  */
}
```

- **Thread-safe random numbers:** each worker thread owns its own `struct drand48_data` and calls `drand48_r()`, which returns a uniform value in [0, 1). The functions `rand()` and `drand48()` keep hidden global state and are not thread-safe.
- **Seeds:** `conn_id` is a server-wide counter incremented for every accepted connection. With the optional `[seed]` argument the generator is seeded with `seed ^ conn_id`, so a run can be repeated with the same sequence of draws per connection. Without it, `time ^ thread id ^ conn_id` is used.

4.3 How the connection is interrupted

- When a draw is below p, the chunk that drew it is **not sent**. The server writes a `FAULT` log record (connection id, session id, topic, file index, offset, chunk index, data bytes sent on the connection). It then calls `shutdown(fd, SHUT_RDWR)` followed by `close(fd)`, updates the statistics and ends the worker thread. The server process keeps running and continues to accept clients.
- Data already sent before the fault is still delivered, followed by end-of-stream. The client detects the failure when `recv()` returns 0 or fails with `ECONNRESET` before the expected end of the file or topic. A receive time-out (`SO_RCVTIMEO`, 15 s) is treated the same way.
- Because faults occur only between chunks, the client's `.part` size after a fault is normally a multiple of 64 KB: 0 if the fault came before the file's first chunk. The resume protocol does not rely on this.

Case	Effect of a fault and recovery
Case 1	The client reconnects after back-off, sends GET again, and the whole topic is re-sent from its first chunk, each chunk with a new draw.
Case 2	The client reconnects and sends RESUME with offset = size of its `.part` file (§5). Only the remaining chunks are sent, each with a new draw.
Case 2 Enhanced	Only the failed connection closes. Its worker re-queues `(file index, .part size)`, reconnects and FETCHes from that offset. The other K − 1 connections continue. Because p is per chunk, the probability of a fault per 64 KB sent is the same as in Case 1 and Case 2.

- `p = 0`: no draw can trigger a fault, and the behaviour is identical to Part I.
- `p = 1`: every draw triggers a fault, so no data chunk is ever sent and no case can complete. Case 1 stops after `max_retries` attempts (default 100). Case 2 and Enhanced stop after `max_retries` consecutive attempts without progress (default 20). All exit with status 2.
- **Retry counting:** in Case 1 `max_retries` counts every attempt. In Case 2 and Enhanced it counts only consecutive attempts in which no new chunk was stored, so a transfer that keeps making progress is never abandoned.

4.4 Expected effect of p

Let N be the number of 64 KB chunks in the topic, $N = \Sigma\lceil \text{file size}/65\,536 \rceil$ over its files, and T its size in bytes. The draws are independent, so:

$$\mathbf{P(\text{no failure over N chunks}) = (1-p)^N}$$

- **Case 1** finishes only on an attempt in which all N draws succeed, which has probability $(1-p)^N$. The expected number of attempts is $1/(1-p)^N$, the expected number of faults is one fewer, and the expected number of chunks transmitted is $(1-p)\big(1-(1-p)^N\big)/\big(p\,(1-p)^N\big)$. Redundant data therefore grows **exponentially** with N and p.
- **Case 2 and Enhanced** send every chunk exactly once, because stored data is kept and a failed chunk was never sent. The expected number of faults, and therefore of reconnections, is $N \cdot p/(1-p) \approx N \cdot p$, which is **linear** in N and p. Data bytes transmitted equal T. Only the control messages of each reconnection and the back-off time are overhead.

Illustration for a topic of N = 1 700 chunks (roughly the size of *cane*). Byte ratios assume chunks of similar size and are approximate:

p (per 64 KB chunk)	**0.0005**	**0.001**	**0.002**	**0.005**	**0.01**
P(no failure over N chunks) = $(1-p)^N$	0.43	0.18	0.033	2.0×10^{-4}	3.8×10^{-8}
Case 1 – expected attempts	2.3	5.5	30	5.0×10^{3}	2.6×10^{7}
Case 1 – expected data transmitted	≈ 1.6 T	≈ 2.6 T	≈ 8.5 T	≈ 590 T	≈ 1.5×10^{6} T
Case 2 / Enhanced – expected faults = reconnects	0.85	1.7	3.4	8.5	17
Case 2 / Enhanced – data transmitted	T	T	T	T	T

With the default `max_retries` = 100, Case 1 is expected to hit the retry limit (recorded as DNF, §6.6) for p ≥ 0.005 on a topic of this size. That is why the Part II sweep uses small per-chunk probabilities.

5 Session management protocol (Case 2)

Session management lets a transfer continue from its last checkpoint after a failure. A **checkpoint** is the tuple (session id, topic, file index, file name, byte offset). The **client is authoritative** for the checkpoint, because only the client knows which bytes were actually stored. The server keeps a session record and uses it to **validate** each checkpoint before resuming.

5.1 Sequence and types of messages

Phase	**Client → Server**	**Server → Client**
1. Start of session	GET <topic>	TOPIC_BEGIN(session_id, file_count, total_bytes), then FILE_HDR / data / FILE_END for each file
2. Normal completion	—	TOPIC_END(file_count, total_bytes). The server marks the session DONE.
3. Failure	(connection lost: recv() = 0, ECONNRESET or time-out)	(fault injected: shutdown + close)
4. Resumption	RESUME(session_id, file_index, offset, topic, name)	RESUME_OK(file_index, offset), then FILE_HDR(start_offset = offset), the remaining bytes, the remaining files and TOPIC_END
5. Invalid checkpoint	RESUME(. . .)	RESUME_REJECT(reason). The client deletes its partial data and returns to phase 1 with GET.

Phases 3–4 repeat as often as failures occur. Figure 5 (§3.7) shows a complete exchange.

5.2 Message formats used by the session protocol

Message	**Fields**	**Purpose**
`TOPIC_BEGIN`	session_id[16] · file_count u32 · total_bytes u64	Creates the session and gives the client its id.

Message	Fields	Purpose
RESUME	session_id[16] · file_index u32 · offset u64 · topic_len u8 · topic str · name_len u16 · name str	Reports the client's checkpoint.
RESUME_OK	file_index u32 · offset u64	Confirms the resume point.
RESUME_REJECT	reason u16 (1–5, see §5.5)	Refuses an invalid checkpoint.
FILE_HDR	file_index u32 · file_size u64 · start_offset u64 · name_len u16 · name str	start_offset tells the client where the following bytes belong.

All messages use the 16-byte header of §3.1.

5.3 Information stored by the server

```
typedef struct {
    uint8_t   id[16];            /* random session id (getrandom)                    */
    int       topic_idx;         /* index into the topic map                         */
    uint32_t  next_file;         /* index of the file currently being sent           */
    uint64_t  file_off_sent;     /* bytes of that file already handed to the socket  */
    uint64_t  total_sent;        /* all data bytes sent in this session (statistics) */
    uint32_t  reconnects;        /* number of RESUMEs accepted (statistics)          */
    int       state;             /* ACTIVE | INTERRUPTED | DONE                      */
    struct timespec last_seen;   /* CLOCK_MONOTONIC, used for expiry                 */
} session_t;

session_t        sessions[MAX_SESSIONS];   /* 256 slots                             */
pthread_mutex_t  sessions_lock;            /* every access holds this mutex         */
```

- The entry is created on GET and updated in memory after every chunk sent. It is marked INTERRUPTED when a fault is injected and DONE on TOPIC_END.
- Entries expire 300 s after `last_seen`, and expired slots are reused.

5.4 Information stored by the client

Item	Location	Updated
Partial file data	output_dir/<topic>/<name>.part	On every write(). Its size is the byte offset of the checkpoint.
Completed file	output_dir/<topic>/<name>	rename(<name>.part, <name>) on FILE_END
Session record	output_dir/<topic>/.session	Written on TOPIC_BEGIN and on every FILE_HDR. It is written to a temporary file and then rename()d, which is atomic. It is deleted after TOPIC_END.

```
# output_dir/gatto/.session   (text, key=value)
session=9f2c4e0b7d11a3c85e6f2a90b4d7c1e3
topic=gatto
file_index=57
file_name=OIP-x7Qm.jpeg
# offset is not stored: it is always stat("OIP-x7Qm.jpeg.part").st_size
```

5.5 Validation of a checkpoint and resumption

#	**Check performed by the server**	**If it fails**
1	The topic exists in the topic map.	RESUME_REJECT (1)
2	file_index $<$ n_files of the topic.	RESUME_REJECT (2)
3	name equals files[file_index].name (detects a changed directory).	RESUME_REJECT (3)
4	offset $\leq$ files[file_index].size.	RESUME_REJECT (4)
5	If the session id is known, the checkpoint is not ahead of what the server sent: file_index $<$ next_file, or file_index $=$ next_file and offset $\leq$ file_off_sent.	RESUME_REJECT (5)

- If all checks pass, the server replies RESUME_OK and sends FILE_HDR with `start_offset = offset`. It then sends only the bytes [offset, file_size), read from that position with `sendfile(out_fd, in_fd, &offset, n)`. The remaining files and TOPIC_END follow. Files before file_index are complete on the client and are **not sent again**, so no stored data is retransmitted.
- If the session id is unknown (e.g. the server restarted or the entry expired), checks 1–4 are enough for a safe resume. The server creates a new entry and continues. A rejected checkpoint never results in wrong data, only in a fresh start.

5.6 Checkpoint update frequency

The byte offset changes with every chunk the client writes, because it is the size of the `.part` file. The `.session` record changes once per file. The server's copy is updated after every chunk sent. The client's checkpoint therefore always matches the data on its disk, and saving it adds no extra I/O. Because the record is on disk, a transfer can also resume after the client program itself is restarted.

Checkpoint ownership trade-off. The table compares the three options for where the checkpoint is kept.

Approach	**Advantage**	**Disadvantage**
Server only	Simple client; server controls resume state	Server may not know exactly which bytes reached client disk
Client only	Client knows exactly what is stored	Server cannot independently validate the checkpoint
Both — chosen	Client provides authoritative stored-byte offset; server validates it	Slightly more state/synchronization

Chosen approach: Both sides maintain checkpoint information, with the client authoritative for the stored byte offset and the server validating the checkpoint before resuming.

6 Planned performance enhancements

The enhancements target Part II Case 2 Enhanced: the socket mode and parallel TCP connections. General optimisations used in every part are listed in §6.3. §6.4–6.7 give the measurement

and experimental plan used to evaluate them.

6.1 Socket mode

- **Main implementation: blocking sockets with one thread per connection.** Part II requires one thread per client, and blocking calls give the simplest correct code. Send and receive time-outs (`SO_SNDTIMEO`/`SO_RCVTIMEO`, 30 s on the server, 15 s on the client) stop an unresponsive peer from blocking a thread forever.
- **Measured variant: non-blocking client.** A single thread drives all K sockets with `O_NONBLOCK` and `poll()`, treating `EAGAIN`/`EWOULDBLOCK` as "retry when ready". It will be compared with the threaded client. `epoll` was considered but only pays off with hundreds of sockets, and here $K \leq 16$.

6.2 Parallel TCP connections

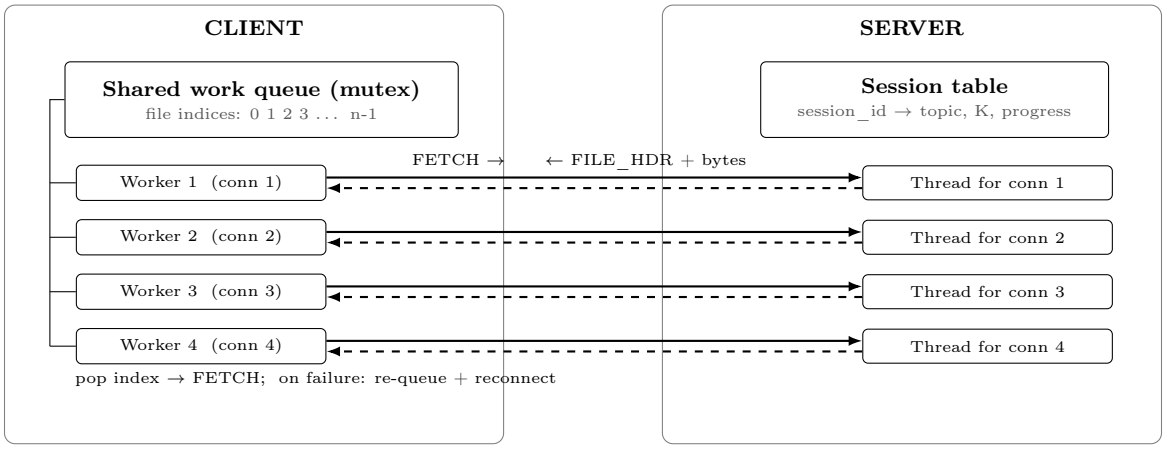


Figure 7: Case 2 Enhanced: K worker connections share one work queue.

Design aspect	Decision and justification
Number of connections	**K = 4** by default (optional client argument, range 1–16). The images are small (tens of KB), so one connection spends part of its time on per-file request/response turnarounds. A few parallel connections keep the link busy, and more mainly compete for the same bandwidth. K = 2, 4 and 8 will be measured.
Work split	**Dynamic, file-level work queue.** The client puts every file index from the MANIFEST into a mutex-protected queue. Each worker repeatedly takes the next index and sends FETCH. Faster connections take more files, which balances load automatically.
Reassembly	Each file is owned by one worker at a time and written to its own `.part` file, which is renamed on FILE_END. Files are independent, so no byte-stream reassembly is needed. For files larger than 8 MB an optional byte-range split is planned: the file is pre-sized with `ftruncate()` and each range is written at its own offset with `pwrite()`.

Design aspect	Decision and justification
Recovery of a failed connection	The affected worker re-queues `(file index, .part size)`, reconnects with back-off and continues with FETCH from that offset. The other $K - 1$ connections are unaffected. The server validates each FETCH with the rules of §5.5.
Completion	The topic is complete when the queue is empty and all K workers are idle (counter + condition variable). The client then deletes `.session` and exits 0.

6.3 Other enhancements (all parts where applicable)

- **Zero-copy transfer:** `sendfile()` moves file data from the page cache to the socket without copying through user space. It also starts at an explicit offset, which the resume logic uses. A `read()` + `sendall()` loop is the fallback.
- **Cached file lists** (§2): no directory scan or `stat()` per request.
- **Fewer small packets:** headers are sent with `MSG_MORE` so the kernel coalesces them with the following file data.
- **64 KB I/O chunks** on both sides. The client writes directly from its receive buffer to the output file.
- **TCP buffer sizing (optional / stretch experiment):** As a stretch experiment, `SO_SNDBUF` and `SO_RCVBUF` may be varied to determine whether larger socket buffers improve throughput on the lab network. The default socket-buffer configuration will be retained for the main experiments.

6.4 Experimental setup

Item	Plan
Machines	Machine **S** runs the server. Machines **A** and **B** run the clients. The topics root is on S's local disk, and every client writes to its own output directory on its machine's local disk (e.g. `/tmp`), so a network-mounted home directory is not measured by mistake.
Recorded for the report	CPU model of S, A and B (`lscpu`), NIC speed (`ethtool`) and gcc version (`gcc -version`).
Synchronised launch	A controller script starts the clients on A and B at the same moment, either via ssh from one controller or by having both launch scripts sleep until the same epoch second. Each client is started in the background with its own output directory, and the script waits for all of them.
Warm-up	Each configuration (Part I and Part II) starts with 1 warm-up run that is discarded, so the page cache is in the same (warm) state for every measured run.
Repetitions	5 measured repetitions per configuration. Every metric is reported as **mean ± standard deviation** over the 5 repetitions.
Fixed seeds	Repetition r uses a fixed seed s_r, passed as the server's `[seed]` argument. The same s_r is used for Case 1, Case 2 and Enhanced, so the cases see comparable fault sequences (§4.2).

Item	Plan
Raw CSV logging	Server, one row per connection: `run_id, case, K, p, seed, conn_id, session_id, topic, start_ns, end_ns, data_bytes_sent, chunks_sent, faulted`. Client, one row per run: `run_id, host, client_id, topic, start_ns, end_ns, useful_bytes, files, attempts, reconnects, status`. All raw CSV files go in `Results/` with the graphs.

6.5 Part I experiments

- **Sweep:** total number of clients N ∈ {1, 2, 4, 8, 16, 32}, split evenly across A and B. N = 1 cannot be split, so that run uses a single client on A.
- **Servers:** each N is run against the single-threaded server and the multi-threaded server.
- **Topic:** all clients request the same topic in every Part I run, e.g. *elefante* for shorter runs, so the runs are comparable.
- **Per configuration:** 1 warm-up run and 5 measured repetitions, reported as mean ± standard deviation.
- **Reported:** server-side aggregate throughput against N for both servers (table and graph `Part1_SingleVsMulti_throughput.png`), plus per-client completion time.

6.6 Part II experiments

- **Sweep:** per-chunk fault probability p ∈ {0, 0.0005, 0.001, 0.002, 0.005, 0.01}.
- **Fixed workload:** one fixed topic for all Part II runs and 4 clients per run, 2 on A and 2 on B. The topic is fixed before the experiments and stated in the report.
- **Implementations:** Case 1, Case 2, and Case 2 Enhanced with K ∈ {2, 4, 8} parallel connections, which is 5 implementations × 6 values of p = 30 configurations.
- **Repetitions:** 5 per configuration, each with its fixed seed s_r, reported as mean ± standard deviation. As in Part I, one warm-up run per configuration is discarded first.
- **Comparison with Part I:** the p = 0 runs are compared with the multi-threaded Part I result at 4 clients.
- **Case 1 DNF:** a Case 1 client that reaches `max_retries` is recorded with status DNF. The number of DNF runs is reported for each p, and DNF runs are excluded from the means.
- **Reported:** overhead and throughput/goodput against p for each case (e.g. `Part2_Case1_overhead.png`, `Part2_Case2_overhead.png`, `Part2_Case2_Enhanced_throughput.png`), and the measured faults and reconnections compared with §4.4.

6.7 Metrics

Metric	Definition
Aggregate throughput (server-side)	Σ data bytes sent ÷ (last connection end − first connection start), measured on the server's single clock with `clock_gettime(CLOCK_MONOTONIC)`.

Metric	Definition
Client-side throughput	Secondary only. Clocks on A and B are not precisely synchronised, so intervals across machines are unreliable. It is used only for per-client figures.
Total bytes transmitted	Σ data bytes sent by the server over all connections of the run, including retransmissions (server CSV).
Useful bytes	Bytes of the topic stored by the clients: T per client, $4 \times$ T per Part II run.
Redundant bytes	Total bytes transmitted $-$ useful bytes.
Overhead %	(Redundant bytes $\div$ useful bytes) $\times$ 100.
Completion time	Per client, from its first connection until TOPIC_END is received, including retry/back-off time. Enhanced has no TOPIC_END (§3.8), so there it ends when the last file of the topic is stored (last FILE_END).
Goodput	Useful bytes $\div$ completion time.
Injected faults	Number of FAULT records in the server log for the run.
Reconnects	From the client log. Case 1 and Case 2: attempts $-$ 1. Enhanced: sum over all worker connections. Apart from time-outs this should equal the number of injected faults, which serves as a consistency check.

7 Additional features beyond the minimum requirements

These features make the implementation robust against the failures we inject, and they produce the data needed for the performance analysis.

Feature	What it does	Why it is included
Logging	The server writes one CSV row per connection: connection id, session id, topic, seed, start/end time, data bytes and chunks sent, and whether a fault was injected. Each FAULT record gives the connection id, session id, file index, offset and chunk index. RESUME events are logged too. The client writes one row per run: attempts, reconnects, bytes received, completion time and status (OK/DNF). A single mutex-protected function does all logging (column list in §6.4).	Provides the raw performance logs for Results/ and allows failure/resume traces to be checked.
Configurability	Optional trailing arguments with defaults: server mode, fault seed, number of parallel connections, retry limit.	Experiments run without recompiling, and the required invocations still work unchanged.
Graceful shutdown	SIGINT/SIGTERM are blocked in all threads. A dedicated thread waits in `sigwait()`, stops accepting connections, lets active transfers finish (up to 5 s), flushes the logs and exits with status 0.	No truncated logs or leaked threads between experiment runs.
Extensibility to new topics	If a request names an unknown topic, the server re-scans `topics_root_dir` under the write lock (at most once per second) before replying 404.	A topic can be added by creating a folder, without restarting the server.

Feature	What it does	Why it is included
Robustness to dead clients	`signal(SIGPIPE, SIG_IGN)` and `MSG_NOSIGNAL`. Writing to a closed socket then returns `EPIPE` instead of terminating the process.	Fault injection creates closed connections on purpose, and the server must survive them.
Path safety	The server resolves topics only through the map and files only through index and name. The client rejects file names containing '/', '..', a leading '.' or control characters.	No read or write can escape the intended directories.
Thread limit	At most 256 concurrent worker threads. Beyond that the server replies `ERROR 503`.	The server stays responsive under overload.
Retry with back-off	Reconnections wait 50 ms, doubling up to 1 s.	Avoids a tight reconnect loop against the server.
Fast restart	`SO_REUSEADDR` on the listening socket.	The server can be restarted immediately between experiments.
Integrity check	The client compares received file sizes and counts with FILE_HDR, MANIFEST and TOPIC_END.	Detects incomplete or inconsistent transfers.

8 Code structure

All four folders share one set of modules: networking, message encoding, topic map, file transfer and logging. Each version differs only in what the server does for one connection (`serve_client()`) and how the client reacts to a failure. The shared modules are copied into every folder, so each folder builds on its own with `make`.

8.1 Module and file structure

```
FinalSubmission_Group_043_006/
├── FinalReport_Group_043_006.pdf
├── README.md                  build/run instructions for every case, fault-injection mechanism
├── Part1/                     Makefile  server.c  client.c  + shared modules
├── Part2_Case1/               ... shared modules + fault.c
├── Part2_Case2/               ... shared modules + fault.c  session.c  checkpoint.c
├── Part2_Case2_Enhanced/      ... shared modules + fault.c  session.c  checkpoint.c  workq.c
└── Results/                   raw CSV logs, *_throughput.png, *_overhead.png

Shared modules (identical in every folder):
    net.h/.c   protocol.h/.c   topic_map.h/.c   transfer.h/.c   log.h/.c
```

8.2 Key functions

Module	Key functions	Responsibility
`net`	`sendall(fd, buf, n)` · `recvall(fd, buf, n)` · `tcp_listen(port)` · `tcp_connect(ip, port)` · `set_timeouts(fd, sec)`	Full-length socket I/O, EINTR handling, socket set-up
`protocol`	`put_u16/u32/u64` · `get_u16/u32/u64` · `send_msg(fd, type, payload, len)` · `recv_hdr(fd, &hdr)` · `recv_payload(fd, hdr, buf)`	Header and payload encoding, decoding and validation

Module	Key functions	Responsibility
`topic_map`	`topic_map_load(root)` · `topic_find(map, key)` · `topic_map_rescan(map)` · `topic_map_free(map)`	Data structures of §2
`transfer`	`server: serve_topic(c, t, from_idx, from_off)` · `send_file_range(c, t, idx, off)` `client: recv_topic(fd, dir, mode)` · `recv_file(fd, dir, hdr)`	File streaming with sendfile(). Calls the fault hook before each chunk
`fault`	`fault_init(c, p, seed, conn_id)` · `should_fail(c)` → true if this chunk's `drand48_r()` draw is < p	Per-chunk fault decision (§4)
`session`	`session_create` · `session_find` · `resume_validate` · `session_update` · `session_expire`	Server-side session table (§5.3, §5.5)
`checkpoint`	`ckpt_load(dir)` · `ckpt_save(dir, ck)` · `ckpt_clear(dir)`	Client-side checkpoint (§5.4)
`workq`	`workq_init(n)` · `workq_pop(&idx, &off)` · `workq_push(idx, off)` · `workq_wait_done()`	Shared file queue of the Enhanced client (§6.2)
`log`	`log_init(path)` · `log_event(fmt, ...)` · `stats_conn(...)`	Thread-safe CSV logging
`server.c`	`main()` · `accept_loop()` · `serve_client(fd)` · `signal_thread()`	Argument parsing, mode selection, thread creation
`client.c`	`main()` · `run_with_retry()` · `worker_main()` (Enhanced)	Argument parsing, retry/resume loop, parallel workers

8.3 How the single-threaded, multi-threaded and fault-tolerant logic fit together

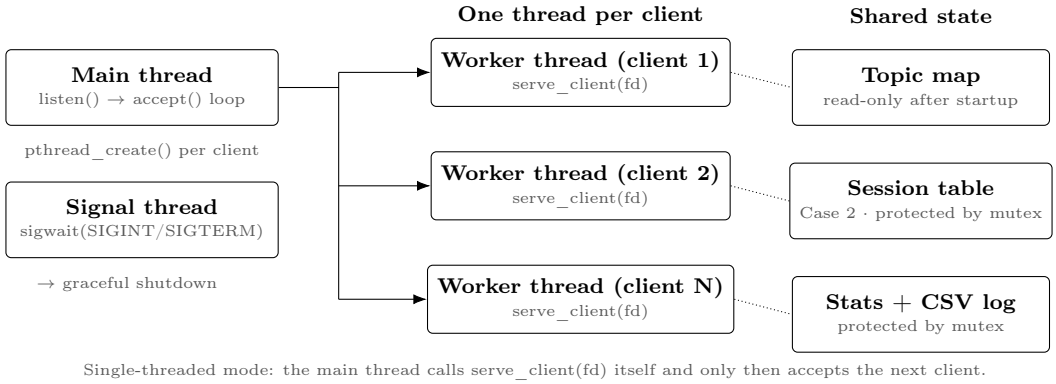


Figure 8: Server thread structure in multi-threaded mode, which all Part II cases use.

- **Single-threaded mode (Part I):** `accept_loop()` calls `serve_client(fd)` directly. The next client is accepted only after the current one has been fully served.
- **Multi-threaded mode (Part I and all of Part II):** for each accepted connection, `accept_loop()` allocates a per-connection context on the heap and starts a detached pthread running `serve_client()`. A separate signal thread handles graceful shutdown. Shared state is either read-only (topic map) or protected by a mutex (session table, statistics and log).
- **Fault-tolerant logic:** added in layers inside `serve_client()` and the client, without

changing the thread model:

Version	serve_client(fd) on the server	Client on connection failure
Part I	read GET → serve_topic(all files)	Not applicable. Any error → exit 1
Case 1	read GET → serve_topic(all files), with should_fail() before every 64 KB chunk	reconnect → GET again, overwrite files
Case 2	read GET or RESUME → resume_validate → serve_topic(from idx, off), with should_fail() before every 64 KB chunk	reconnect → RESUME with the .part-size checkpoint
Case 2 Enhanced	read MANIFEST_REQ → MANIFEST; or loop { read FETCH → validate → send_file_range } with should_fail() before every 64 KB chunk	only the failed worker re-queues its file, reconnects and FETCHes from the offset

8.4 Build

```
CC     = gcc
CFLAGS = -std=gnu11 -Wall -Wextra -Werror -O2 -pthread
OBJS   = net.o protocol.o topic_map.o transfer.o log.o     # plus fault.o, session.o ... per
all: server client
server: server.o $(OBJS) ; $(CC) $(CFLAGS) -o $@ $^
client: client.o $(OBJS) ; $(CC) $(CFLAGS) -o $@ $^
clean: ; rm -f *.o server client
```

`-Werror` turns any compiler warning into a build failure, so the submitted code builds without warnings. During development we also build with `-fsanitize=address,undefined` and `-fsanitize=thread` to catch memory errors and data races.

9 Design thought process: alternatives considered

For each case we compared the main alternatives before choosing an approach. The table is grouped by case, as the assignment asks. The first group covers decisions that apply to every case.

Decision	Alternatives considered	Chosen approach and reason
General (all cases)		
Language	Python · C++ · C	**C with pthreads.** The assignment is built around pthreads and clock_gettime(). C gives direct control of every system call and adds no interpreter dependency on the grading machine.
Message framing	Newline-terminated text (as in HTTP/FTP) · delimiter bytes · length-prefixed binary header	**Length-prefixed binary header.** Image data can contain any byte value, so delimiters are unsafe. An explicit length tells the receiver exactly where each message ends. The request is still the text "GET <topic>".
Part I		
Topic map	Re-scan the directory on every request · hash table · sorted array	**Sorted array with cached file lists.** With about 10 topics, binary search is as fast as hashing and simpler. The cache avoids disk scans and gives stable file indices for Case 2.

Decision	Alternatives considered	Chosen approach and reason
Concurrency model	Thread per client · thread pool · single-threaded event loop (epoll)	**Thread per client** (also required by Part II). It is the simplest correct design for tens of clients. A pool bounds resources but adds queueing delay. An event loop only pays off with thousands of connections. A cap of 256 threads limits resource use.
Single vs multi-threaded	Two separate programs · compile-time flag · runtime argument	**Optional runtime argument.** The grader requires exactly one server executable per folder, and one build can then run both experiments.
Part II – Case 1		
Where the fault decision is made	One draw per connection with a random cut point · separate "killer" thread · independent draw before every 64 KB chunk	**Independent draw before every 64 KB chunk**, inside the worker's file-send loop. The decision point is well defined, uses only the worker's own state, and gives p one precise meaning in every case: the fault probability per 64 KB chunk. The chance of a clean transfer, $(1-p)^N$, shows directly how exposure to failure grows with the amount of data, which is the effect Case 1 vs Case 2 is meant to measure. A per-connection draw with a random cut point hides that effect and needs each connection's expected byte count, which differs in Enhanced. A killer thread would race with the worker on the same socket. The cost is that Case 1 degrades exponentially with N, which the retry cap (DNF) and the small p range of §6.6 handle.
Random number generator	rand() / drand48() · rand_r() · drand48_r() with per-thread state	**drand48_r().** rand() and drand48() keep hidden global state and are not thread-safe. rand_r() is obsolescent in POSIX. drand48_r() with a per-thread drand48_data is thread-safe and can be seeded for repeatable experiments.
How the connection is interrupted	Abortive close (RST via SO_LINGER) · shutdown() followed by close()	**shutdown + close.** Deterministic: the chunk that drew the fault is not sent, the client receives every chunk sent before it and then sees end-of-stream, so byte accounting is exact.
Restart behaviour	Delete the output directory and restart · overwrite files in place	**Overwrite in place (O_TRUNC).** Simpler, and it still retransmits the whole topic as Case 1 requires.
Part II – Case 2		
Who stores the checkpoint	Server only · client only · both	**Both, with the client authoritative.** The server knows the bytes it sent, not the bytes the client stored. The client's .part size is exact, and the server's record lets it reject impossible checkpoints.
Checkpoint frequency	After every file · every N KB · continuously	**Continuously at no cost.** The .part size is the offset, and .session is rewritten only when the file index changes.

Decision	Alternatives considered	Chosen approach and reason
Invalid checkpoint	Close the connection · resume from the start of the current file · reject and let the client restart	**RESUME_REJECT with a reason code.** It never delivers wrong data, and the client falls back to a clean GET.
Part II – Case 2 Enhanced		
Work split	Static round-robin · byte ranges within files · dynamic file queue	**Dynamic file queue.** Files are small, so splitting inside a file adds overhead without benefit. A dynamic queue balances load and makes recovery simple (re-queue the file).
Number of connections	Fixed 2 · fixed 8 · configurable	**Configurable, default 4.** The best value depends on the lab network and will be chosen from measurements of K = 2, 4 and 8.
Socket mode	Blocking + threads · non-blocking + poll/epoll	**Blocking + threads** as the main design, for clarity and correctness. A non-blocking poll() client is implemented as a measured comparison.